DE 424: Wumpus Trajectory Tracking Model

Amy McDermott (26911264)

May 14, 2026

Declare that this work presented is your own work. Mention any sources that contributed to work or report
I declare that this report is my own work. This report was written using LaTeX and thus Gemini was used to assist in certain formatting. This includes help to format code listings, mathematical equations and image display. The mathematical derivations and interpretation of results are my own work.

ECSA graduate attributes

Discuss how the GAs were assessed in the module + which parts of the report this is documented. State HOW and WHERE they were satisfied This section may (and should) be brief, but should discuss all aspects of the GAs being assessed. For example, GA1 partially assesses your ability to research literature as part of problem solving, which means that your report must mention and cite (at least) a few sources - can include prescribed resources. Please consult the module framework for a more detailed description of these two GAs as well as how they are assessed in this module.

GA 1: Problem solving

GA 2: Application of scientific and engineering knowledge

1 Problem Description

This project addresses an object tracking task within a discrete grid world inhabited by a latent entity known as a wumpus. The objective is to determine the trajectory of the wumpus across multiple time steps based on inaccurate measurements. At each time step the wumpus attempts to move to an adjacent cell (up, down, left, or right) with equal probability. If the target cell is within the grid boundaries, the wumpus transitions to that location; otherwise it remains in its current position. The exact location of the wumpus is never directly observed. Instead, every cell in the grid generates a binary detection value at each time step, parameterised by:

- True Positive (p_w): The probability that the cell containing the wumpus correctly generates a detection.
- False Positive (p_c): The probability that a cell not containing the wumpus generates an incorrect detection (clutter measurement).

Given the stochastic grid detection data across multiple time steps, the task is to design, implement and evaluate a Probabilistic Graphical Model (PGM) solution. This involves constructing a model that accurately captures the wumpus's transition and observation logic to infer it's latent trajectory across various datasets.

2 Solution Design

The solution was developed iteratively to accomodate the increasing complexity and varying characteristics of each dataset. The PGM design process followed a structured methodology: defining the random variables, defining the model, and constructing the model factors. This process is detailed below for each attempted dataset. (note: this section must show model structure and show all factors. State and motivate all design choices)

2.1 Dataset 1

The initial model was designed for a 5×5 grid over a series of $T = 10$ time steps. In this case, the parameters $p_w = 0.95$ and $p_c = 0.05$ are known constants.

2.1.1 Random Variable Definition

A critical design choice was made to represent the wumpus's location using a single latent RV, W^t, instead of seperate X^t and Y^t coordinates. The grid is mapped to a 1-dimensional vector where each cell index $i \in \{0, \ldots, 24\}$ corresponds to a unique (x, y) coordinate. This approach offers several advantages. Primarily, it enhances inference efficiency by reducing the total number of RVs within the cluster graph, which leads to more efficient message passing and overall faster convergence. Furthermore, it reduces complexity by simplifying factor construction and avoiding the complexities of coordinating the dependencies between spatial coordinates. The wumpus's movement constraints are more easily enforced within a transition matrix; for example, the probability of illegal moves, such as diagonal steps, can be explicitly set to zero within one factor. The RVs are defined as follows:

- $W^t \in \{0, 1, \ldots 24\}$: The latent position of the wumpus at time t.
- D_i^t in {0,1}: The binary detection status of cell i at time t, where 1 indicates a detection.

2.1.2 Model Definition

The wumpus tracking task is formulated as a Hidden Markov Model (HMM), a framework used to infer a sequence of hidden states from observable outputs [1]. The latent trajectory of the wumpus $(W^0, W^1, \ldots, W^T)$ is modeled as a first-order Markov process, $P(W^t \mid W^{t-1})$, where every future state depends on the current state, not on the sequence of states preceding it. The observable detection variables are emissions that are generated from each state $p(D_l^t | W^t)$. A HMM is a subclass of Bayesian Networks, used for sequential data. The model is illustrated below in Figure 1.

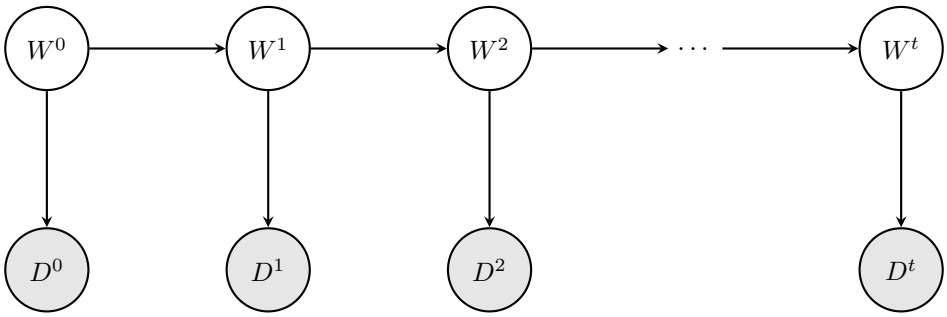


Figure 1: Bayesian Network structure of the HMM tracking solution.

2.1.3 Factor Definition

The model is governed by two primary factors. The first factor models the transition model of the wumpus over time as a first-oder Markov process $P(W^t \mid W^{t-1})$. This factor captures the entity's movement across adjacent cells, where the wumpus attempts to move up, down, left, or right with an equal probability of 0.25 per direction. To enforce the physical boundaries of the grid, the wumpus remains in its current position if it attempts to move outside the grid world. Consequently, the probability of the wumpus remaining stationary $(P(W^t = i \mid W^{t-1} = i))$ is determined by its proximity to the boundaries:

- Interior cells: The wumpus always moves to an adjacent cell with a probability of 0.25. Thus, the probability of remaining in place is 0.
- Edge cells: With the wumpus having one direction blocked, the probability of remaining in place is 0.25.

- Corner cells: With the wumpus having two blocked directions, the probability of remaining in place increases to 0.5.

For a 5×5 grid, the stochastic wumpus trajectory results in a sparse 25×25 transition matrix that ensures that the inferred wumpus movement adheres to the constraints of the environment. This transition factor is invariant across all time steps $t \in \{1, \ldots, T\}$ since the wumpus's rules of movement do not change as time progresses.

Table 1: Illustrative sample of the Transition Probability Factor $P(W^t \mid W^{t-1})$ for a 5×5 grid.

W^{t-1}	W^t	$P(W^t \mid W^{t-1})$	**Description**
0	0	0.50	Stay (Blocked Up and Left)
0	1	0.25	Valid Move (Right)
0	5	0.25	Valid Move (Down)
0	6	0.00	Illegal Move (Diagonal)
12	12	0.00	Stay (No Boundaries Hit)
12	13	0.25	Valid Move (Right)
...	...	...	...

Secondly, a factor that links the latent state W^t to the observed grid-wide detection data $\mathbf{D}^t$ is designed $P(\mathbf{D}^t \mid W^t)$. This factor is parameterised by the detection probability p_w and the clutter probability p_c, which accounts for sensor inaccuracies across all cells in the grid. In the context of HMMs, this factor defines the likelihood of a specific observation being generated from a particular hidden state, otherwise known as an emission probability.

Table 2: Illustrative sample of the Detection Factor for a single cell i at time t.

W^t	D_i^t	$P(D_i^t \mid W^t)$	**Description**
i	1	p_w (0.95)	True Positive (Correct Detection)
i	0	$1 - p_w$ (0.05)	False Negative (Missed Detection)
$j \neq i$	1	p_c (0.05)	False Positive (Clutter)
$j \neq i$	0	$1 - p_c$ (0.95)	True Negative (Correct Non-Detection)

2.1.4 Cluster Graph and Inference

The topology of an HMM naturally forms an acyclic graph. (source). Due to this, the Bayesian Network shown in Figure 1 is converted to a Junction Tree cluster graph using the built-in functionality of the **emdw** library. This structure guarantees exact inference through a single forward-backward message-passing schedule, resulting in a balance between computational efficiency and inference accuracy. Due to the tree structure, the complexity scales linearly, $O(T)$, with the number of time steps. The factors and cluster graph are shown below:

- $P(W^t \mid W^{t-1})$ is assigned to a cluster with the scope $\{W^{t-1}, W^t\}$.
- $P(\mathbf{D}^t \mid W^t)$ is assigned to a cluster with scope $\{W^t\}$.

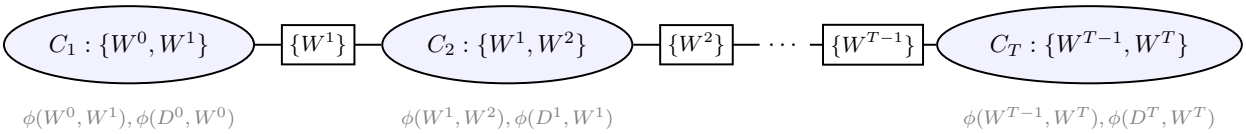


Figure 2: Cluster graph for exact inference.

For the trajectory inference task, Maximum A Posteriori (MAP) inference using the Max-Product algorithm was selected over Marginal inference. This choice is motivated by the fact that MAP inference

computes the most likely trajectory $(W^0, W^1, \ldots, W^T)$ which results in a path that is globally consistent and adheres to the wumpus's movement constraints as defined in the transition model. The MAP estimates represent a valid movement of the wumpus through the grid world. In contrast, Marginal inference using the belief-propagation algorithm identifies the most likely cell for the wumpus at each discrete time step independently. This may lead to a sequence of locations that are infeasible if certain cells have high local probabilities.

2.2 Dataset 2

Extends upon dataset1 - we can see if model can expand for larger grid sizes. Dataset 2 uses a 20×20 grid. This results in 400 possible states. The transition factor for Dataset 2 will therefore be a 400×400 matrix. $O(T)$ scales.

3 Code Implementation

3.1 Define the Random Variables

The model uses two primary vectors for RV management: `W_rvs` for the latent wumpus positions and `D_rvs` for observed detections. Storing the random variable IDs in these vectors allows efficient looping during the factor construction phase, as specific time dependent RVs (W^{T-1} and W^t) can be accessed directly via their vector indices. `D_rvs` is a 2D vector which indexes one RV for each time step and cell location.

```
vector<T> W_rvs(T_max);
vector<vector<T>> D_rvs(T_max, vector<T>(25));
```

Listing 1: Declaration of latent and observable random variable vectors.

3.2 Implement Model Factors

To ensure that the tracking solution computes the Maximum A Posteriori (MAP) trajectory rather than the marginal probabilities, the factors were instantiated with specific operators. During the construction of both transition and detection factors, specialised pointers for marginalization (`DiscreteTable_MaxMarginalize`) and normalization (`DiscreteTable_MaxNormalize`) were passed to the `DiscreteTable` (`DT`) constructor. By defining the factors this way, the functions called by the cluster graph change their behavior: the `marginalize()` operation performs maximisation instead of summation, and the `normalize()` operation scales the factor such that its maximum value is 1.

3.2.1 Implementation of the Transition Factors

The transition model is implemented by iterating through each time step and constructing a transition factor $P(W^t \mid W^{t-1})$ that connects every current state to its previous. The implentation logic follows a few key steps. Firstly, within each iteration of the temporal loop, the unique IDs for the current and previous wumpus positions are retrieved from the `W_rvs` vector. Because the grid is represented using a 1D index (0 to 24), the code converts these indices back into (x, y) coordinates to facilitate the check for valid adjacent moves.

```
int x = i % width; // x-coordinate
int y = i / width; // y-coordinate
```

Listing 2: Conversion of grid indices to (x, y) coordinates

Coordinate offsets (dx and dy) are used to determine all potential subsequent wumpus location. To handle the grid boundaries, the proposed wumpus position is checked if it stays withing the grid dimensions. If a move is valid, it is assigned a probability of 0.25; and if a move would go out of bounds, the 0.25 probability is instead accumulated. This ensures that the probability of staying in a cell correctly reflects how many boundaries that the cell is touching (e.g., 0.5 in a corner).

```
// Define possible grid moves (up, down, left, right)
int dx[] = {0, 0, -1, 1};
int dy[] = {1, -1, 0, 0};
for (int move = 0; move < 4; move++) {
  int new_x = x + dx[move]; // wumpus new x-coordinate
  int new_y = y + dy[move]; // wumpus new y-coordinate
  if (new_x >= 0 && new_x < width && new_y >=0 && new_y < width) {
    // Valid transition
    int new_i = new_y * width + new_x; // Convert back to cell index
    transitionProbs[{i, new_i}] = 0.25;
  } else {
    // Invalid transition
    stay_prob += 0.25; // Accumulate probability of staying in cell
  }
}
transitionProbs[{i, i}] = stay_prob;// Prob of wumpus staying in the same cell
```

Listing 3: Transition factor logic.

A `std::map` is used to store only the non-zero probabililities to prevent creating a massive 25×25 table with many zeros which consequently improves computational performance. These sparse entries are then used to instantiate an `emdw DT` called `ptrTransition`, which is then stored in a vector of factor pointers for later use in the cluster graph.

```
transitionFactors.push_back(ptrTransition);
```

Listing 4: Vector of transition factor pointers

3.2.2 Implementation of the Detection Factors

The relationship between the latent wumpus position and the observed detection data $P(\mathbf{D}^t \mid W^t)$ is implemented through a nested looping structure that generates a detection factor for every cell at every time step. An outer loop is utilised to iterate through each time step t and an inner loop to iterate through every grid cell `loc`. This ensures that each cell in the grid world generates its own binary observation variable D^t_{loc} dependent on the wumpus's latent position W^t. For each cell, the probability of a detection (1) or no detection (0) is evaluated based on whether the wumpus is currently occupying that specific cell.

- $w\_pos == loc$ (True detection): The wumpus is in the cell. The probability of a correct detection is assigned the parameter p_w, while a missed detection is $1 - p_w$.
- $w\_pos \neq loc$ (Clutter measurement): The wumpus is not in the cell. The probability of a detection is p_c, and a correct non-detection is $1 - p_c$.

```
// loc is the location of the specific cell we are building a factor for
for (int loc = 0; loc < num_cells; loc++) {
  int D_curr = D_rvs[t][loc]; // Extract index of D RV at time t and grid location
  map<vector<T>, FProb> detectionProbs;
  // w_pos is the latent position of the wumpus
  for (int w_pos = 0; w_pos < num_cells; w_pos++){
    if (w_pos == loc) {
      // Detection location is equal to the wumpus position
      detectionProbs[{1, w_pos}] = pw; // Detection
      detectionProbs[{0, w_pos}] = 1.0 - pw;  // Missed detection
    } else {
      detectionProbs[{1, w_pos}] = pc; // Clutter
      detectionProbs[{0, w_pos}] = 1.0 - pc; // No detection
    }
  } // end of wumpus location loop
```

```
}
```

Listing 5: Detection factor initialisation.

The factor scope is defined as $\{D^t\_loc, W^t\}$ which is extracted from the W_rvs and D_rvs vectors. By using a `std::map` to store the `detectionProbs`, the sparse implementation remains efficient. These probabilities are used to instantiate DT factors which are then stored in a `detectionFactors` vector.

```
detectionFactors.push_back(ptrDetections);
```

Listing 6: Vector of detection factor pointers

3.3 Cluster Graph Construction

This stage involves assembling the defined factors into a cluster graph to perform inference. This process is handled using `emdw` functionality. Firstly, all factors are collected into a single vector of pointers (`factorPtrs`), and an observation map (`obsv`) is created to store all evidence of the observed RVs. The binary detection data is loaded from external dataset files and mapped to the corresponding D^t_{loc} variable IDs. This ensures that the inference procedure is conditioned on the actual data provided in the datasets. A `ClusterGraph` object is instantiated using the `BETHE` method which constructs a Junction Tree. The `loopyBP_CG` function executes exact inference on the tree structure. Once message passing has converged, the cluster graph contains the posterior information required to determine the most likely state of the wumpus at each time step.

```
ClusterGraph cg(ClusterGraph::BETHE, factorPtrs, obsv);
map<Idx2, rcptr<Factor> > msgs;
MessageQueue msgQ;
unsigned nMsgs = loopyBP_CG(cg, msgs, msgQ);
```

Listing 7: Cluster graph creation and message passing

3.4 Extraction of the MAP Estimates

The result of MAP message passing is a set of max-marginal beliefs. The most likely local combination of each max-marginal belief is consistent with the most likely global combination. For every time step t, the cluster graph is queried using `queryLBP_CG` for the belief of the latent variable W^t. This belief represents the maximum joint probability of the entire sequence passing through that specific state at time t. A nested loop then iterates through all possible grid cells to identify the cell that yields the highest value in the max-marginal factor. This identifies the specific position, at that time step t, that belongs to the overall most likely trajectory.

```
rcptr<Factor> beliefT = queryLBP_CG(cg, msgs, {W_rvs[t]})->normalize(); // Query
for (int row = 0; row < R; row++) {
  for (int col = 0; col < C; col++) {
    // Calculate the cell index and extract likelihood
    unsigned int cell_idx = row * C + col;
    double likelihood = beliefT->potentialAt({W_rvs[t]}, {(T)cell_idx});
    if (likelihood > max_likelihood) {
      max_likelihood = likelihood;
      best_cell = cell_idx;
    }
  }
}
```

Listing 8: Extraction of MAP estimates from max-marginal beliefs.

To facilitate the model evaluation phase, each inferred cell index is converted back into Cartesian (x, y) coordinates and exported to an external text file. Persisting the data in this manner ensures that the

inferred trajectory remains structurally consistent with the ground-truth datasets. Once this process has been completed for every time step t, the inference procedure has identified the single most likely sequence of locations for the wumpus across all timesteps.

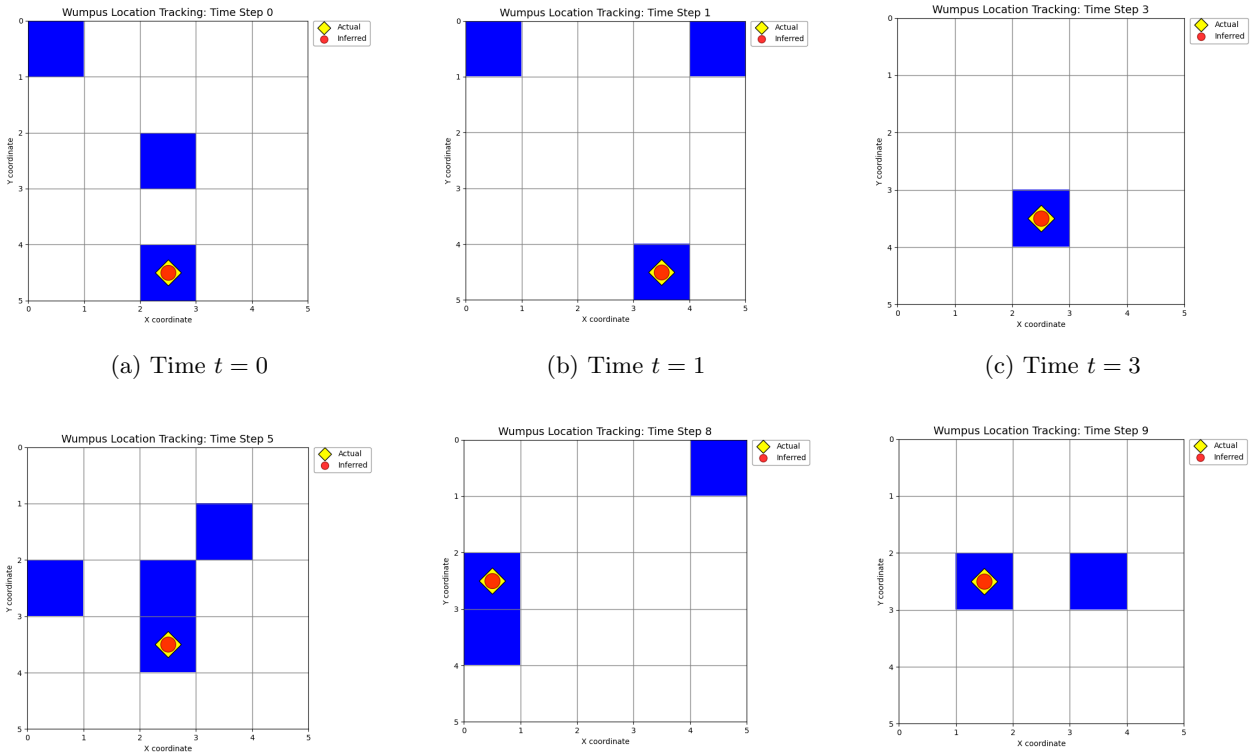
3.5 Dataset 2

updated how to create locationDom updated all parameters (num_cells) etc changed reading in data function to account for the different naming convention

A brief description of how you processed the output of your solution, the presentation of your results, and an analysis thereof. You should report appropriate qualitative and quantitative results for all the datasets.

4 Analysis of Results

Model performance is qualitatively assessed using a Python script to visualise the wumpus's path on the grid world. The detection data is overlayed with the inferred trajectory and ground-truth data, allowing for a clear visual comparison of the results. Figure 3 displays these results for Dataset 1 at various time steps, where the yellow star represents the ground-truth wumpus location and the red dot represents the location inferred by the MAP algorithm. The corresponding visualisation for Dataset 2 is shown in Figure 4, where the model scales to a larger environment.

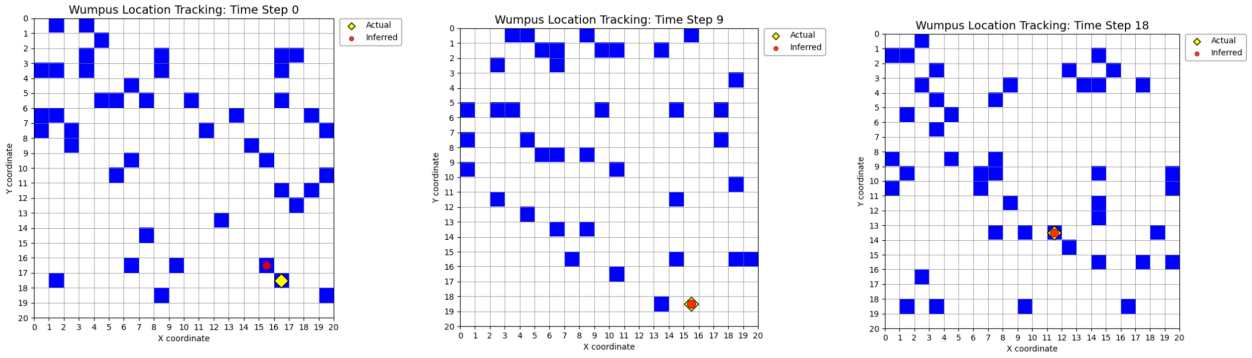


(a) Time $t = 0$ (b) Time $t = 1$ (c) Time $t = 3$

(d) Time $t = 5$ (e) Time $t = 8$ (f) Time $t = 9$

Figure 3: Dataset 1 tracking performance across select time steps.

A quantitative evaluation is used to objectively analyse the performance and scalability of the models. The primary metrics utilised include: Accuracy, Neighbourhood Accuracy, Root Mean Square Error and Mean Manhattan Distance (MMD). Accuracy represents the percentage of times steps where the inferred MAP location W^t matches the ground-truth coordinates. To quantify classification deviations, the RMSE



(a) Time $t = 0$ (b) Time $t = 9$ (c) Time $t = 18$

Figure 4: Dataset 2 tracking performance on the 20×20 grid environment.

was calculated to measure the average Euclidean distance between the predicted and actual positions.

$$RMSE = \sqrt{\frac{1}{T}\sum_{t=0}^{T-1}[(x_t - \hat{x}_t)^2 + (y_t - \hat{y}_t)^2]} \tag{1}$$

Neighborhood Accuracy and MMD obtain a more naunced understanding of the model's tracking ability, especially in the more complex environments of Dataset 2. The Neighborhood Accuracy measures the percentage of time steps that infer the location within one grid step of the actual position. Since the classes represent a physical grid, an inference that is one cell removed from the ground truth is more valuable than one on the other side of the map. Lastly, the MMD provides an intuitive error measurement that represents the average number of moves separating the MAP estimate from the wumpus's true location.

$$MMD = \frac{1}{T}\sum_{t=0}^{T-1}(|x_t - \hat{x}_t| + |y_t - \hat{y}_t|) \tag{2}$$

Table 3: Quantitative Results for Wumpus Tracking (Datasets 1 & 2).

Dataset	**Acc. (%)**	**N-Acc. (%)**	**RMSE**	**MMD**	**Remarks**
Dataset 1	100.00%	100.00%	0.0000	0.00	Perfect tracking.
Dataset 2	85.00%	85.00%	0.5477	0.30	Deviations at $t \in \{0, 5, 11\}$.

**Acc: Exact Match Accuracy; N-Acc: Neighborhood Accuracy (within 1 cell);*
RMSE: Root Mean Square Error; MMD: Mean Manhattan Distance (steps).

References

[1] Daniel Jurafsky and James H. Martin. *Hidden Markov Models.* 2023. Appendix A.